

TEAM GUIDE — Linux File System Explorer (A to Z)

Course: Operating System and System Programming (25CS2104E) **Team 14, Section 12** — M. Shiva (2520090135), N. Sainath Reddy (2520090115)

This ONE document explains the ENTIRE project: the concepts, the code line by line, how to run it, how to demo it, and how to answer viva questions. Read it top to bottom. If you understand this document, you understand 100% of the project.

PART 1 — THE BIG PICTURE (explain this in 30 seconds)

What we built: A menu-driven command-line program written in C that manages files and directories — but instead of using easy library functions, it talks to the Linux kernel DIRECTLY using low-level system calls: `open()`, `read()`, `write()`, `close()`, `lseek()`, `stat()`, `opendir()`, `readdir()`, `mkdir()`, `rmdir()`.

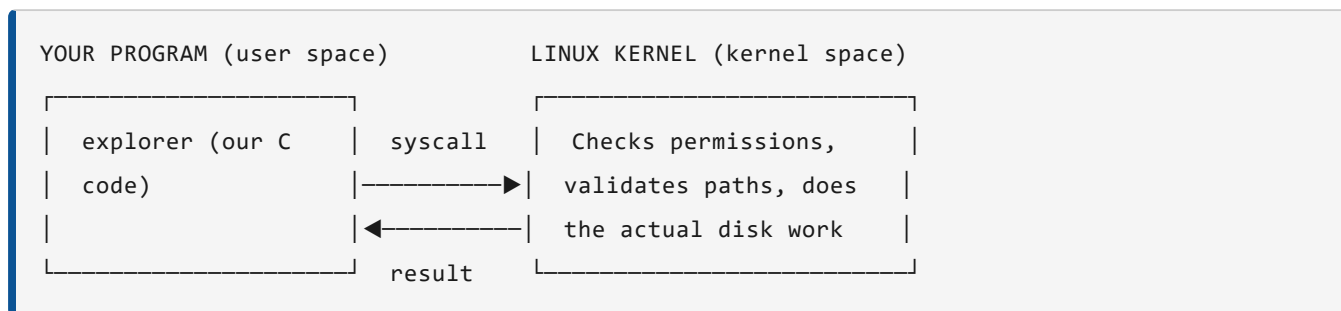
The one-line answer for viva: "Our program is a File System Explorer that demonstrates how a user program communicates with the Linux kernel through POSIX system calls — every file and directory operation in the menu goes through a real system call, with full error checking using `errno`."

Problem it solves (memorize this paragraph): Many file-management applications hide the low-level operations performed by the operating system. This makes it difficult for students to understand how Linux handles files, directories, metadata, permissions, and file descriptors. Our solution is a simple command-line File System Explorer that directly uses Linux/POSIX system calls to perform common file and directory operations and demonstrate communication between a user program and the Linux kernel.

PART 2 — THE 6 CONCEPTS YOU MUST KNOW (the theory)

2.1 What is the Kernel?

The **kernel** is the core of the Linux operating system. It is the only component allowed to touch hardware: disk, memory, CPU scheduling.



Normal programs (user programs) are NOT allowed to touch the disk directly — that would be a security disaster. Whenever a program needs the disk, it must **ask the kernel** politely. That "asking" is a **system call**.

2.2 What is a System Call?

A **system call** is a function that transfers control from your program (user mode) into the kernel (kernel mode) to perform a privileged operation, then returns the result.

- Example: when our code calls `open("notes.txt", O_RDONLY)`, the CPU switches to kernel mode, the kernel finds the file, checks our permissions, prepares it, and hands back a number.

- It is more expensive than a normal function call (mode switch), which is exactly why we demonstrate it — it is THE bridge between user program and kernel.

- **Viva trap:** `printf()` and `fopen()` are LIBRARY functions (in libc) — they are not system calls, but they internally use system calls (`write()`, `open()`). Our project calls the system calls directly.

2.3 What is a File Descriptor?

A **file descriptor (fd)** is a small non-negative integer the kernel gives your process when you open a file. It is an index into the kernel's table of open files for your process — like a ticket number at a counter.

- 0 = standard input (keyboard)
- 1 = standard output (screen)
- 2 = standard error

Those three are opened automatically for every process. So the **first file our program opens usually gets fd 3**. We never hard-code 3 — we print whatever the kernel returns:

```
fd = open(path, O_RDONLY);
printf("File Descriptor: %d\n", fd);    /* prints 3, 4, 5... as kernel assigns */
```

2.4 What is errno?

Every system call reports failure the same way:

1. It **returns a special value** — usually `-1` (or `NULL` for `opendir()`).
2. It **sets the global variable** `errno` — an integer error code.

`errno` alone is useless to a human (it's just a number like 2), so we convert it to text:

```
print_errno("Unable to open file.");  
/* prints: [ERROR] Unable to open file.  
          Reason: No such file or directory.      <- from strerror(errno) */
```

Common `errno` codes we handle (memorize 4–5):

Code	Meaning	When you'll see it
ENOENT	No such file or directory	Reading a file that doesn't exist
EACCES	Permission denied	Opening a file you're not allowed to
EISDIR	Is a directory	Trying to <code>read()</code> a directory
ENOTEMPTY	Directory not empty	<code>rmdir()</code> on a folder with files
ENOTDIR	Not a directory	<code>chdir()</code> to a file name
EINVAL	Invalid argument	<code>lseek()</code> to a negative offset

2.5 What is an inode?

Every file on a Linux filesystem has an **inode** — a data structure that stores the file's METADATA (size, permissions, owner, timestamps) and pointers to its data blocks on disk. **The file NAME is not stored in the inode** — names live in the directory, which maps name → inode number.

That's why in Option 9 we show the inode number from `stat()`:

```
printf("Inode: %lu\n", (unsigned long)file_stat.st_ino);
```

Viva gold: this is also why `rename()` is instant — it only changes the name→inode mapping, it doesn't move any data. (We demonstrate exactly this in Option 12.)

2.6 open() vs fopen() — the classic question

	<code>open()</code> (we use)	<code>fopen()</code> (we avoid)
Type	System call (thin POSIX wrapper)	C library function
Returns	File descriptor (int)	FILE* pointer
Buffering	None — unbuffered, direct	Buffered inside your process
Level	Low-level / kernel	Higher-level, portable

We deliberately use `open()/read()/write()` because the **whole point of the project is to show the kernel interaction**, which `fopen()` hides.

PART 3 — THE JOURNEY OF ONE SYSTEM CALL (explain this on the board)

When our program calls `open("notes.txt", O_RDONLY)`:

1. Our C code calls `open()` (user mode)
↓
2. C library wrapper puts the syscall number + arguments in registers
↓
3. Special instruction switches CPU to KERNEL MODE (mode switch)
↓
4. Kernel: resolves the path "notes.txt" (starting from our current directory)
↓
5. Kernel: checks permissions on every directory component and the file itself
↓
6. Kernel: finds the inode, creates an "open file description" (with file offset = 0), assigns the lowest free fd number → 3
↓
7. CPU switches back to USER MODE and `open()` returns 3 to our code
↓
8. On failure instead: returns -1 and sets `errno` (e.g. `ENOENT`)

PART 4 — PROJECT STRUCTURE (what file does what)

```
LinuxFileSystemExplorer/
├─ src/
│   ├── main.c           ← menu loop + dispatch (the "receptionist")
│   ├── file_operations.c ← open/read/write/close/lseek/copy/rename
│   ├── directory_operations.c ← opendir/readdir/mkdir/rmdir
│   ├── metadata.c       ← stat/lstat, permissions, timestamps, inode
│   ├── navigation.c     ← getcwd/chdir
│   └─ utils.c           ← safe input, menus, colors, error printing
├─ include/              ← headers: the "contract" of each module
│   ├── file_operations.h │── directory_operations.h │── metadata.h
│   ├── navigation.h      └─ utils.h
├─ build/                ← created by make (object files + binary)
├─ tests/
│   ├── test_cases.md    ← 18 tests with REAL observed results
│   ├── demo_input.txt   ← scripted inputs for automatic demo
│   └─ run_demo.sh       ← runs the demo in a scratch folder
├─ docs/                 ← this guide + formal report
├─ README.md             ← formal project documentation
├─ Makefile              ← build automation
└─ .gitignore
```

Why split into modules? (viva answer):

1. **Separation of concerns** — each file owns one topic (files / directories / metadata / navigation / shared utilities).

1. **Teamwork** — different members can work on different files without conflicts.

1. **Maintainability** — a bug in directory listing can't corrupt file writing.

2. **Headers as contracts** — `file_operations.h` declares WHAT the module offers; `file_operations.c` implements HOW. Other modules include only the header, never the source.

Every `.c` file starts with:

```
#define _POSIX_C_SOURCE 200809L
```

Why: under strict `-std=c11`, the compiler hides POSIX extensions like `lstat()` and `readlink()`. This macro says "give me the POSIX.1-2008 API" — it must come BEFORE all includes.

PART 5 — HOW THE PROGRAM RUNS (main.c walkthrough)

5.1 The skeleton

```
int main(void)
{
    ui_init();                /* enable colors only if output is a terminal */
    print_welcome();          /* ASCII banner */

    while (running) {
        print_menu(current_dir_or_fallback(cwd, sizeof(cwd)));
        printf("Enter your choice: ");

        if (read_line(line, sizeof(line)) == NULL) { ... } /* EOF = quit */
        if (!parse_int(trim(line), &choice)) { ...error... } /* "abc" etc. */
        if (choice < 1 || choice > MENU_EXIT) { ...error... }
        if (choice == MENU_EXIT) { running = 0; break; }

        menu_actions[choice](); /* ← dispatch! */
    }
}
```

5.2 The dispatch table (important structure!)

Instead of a giant `switch`, we use an **array of function pointers**:

```
typedef void (*menu_action_t)(void);

static const menu_action_t menu_actions[] = {
    NULL,                /* index 0 unused */
    run_show_current_directory, /* 1 */
    run_list_directory,    /* 2 */
    ...
    NULL                 /* 13 = Exit, special case */
};
```

When you type **4**, the program runs `menu_actions[4]() → run_create_file() → op_create_file()` in `file_operations.c`.

Viva answer for "why function pointers?": it maps each menu number to its handler cleanly, keeps `main()` short, and adding a new option means adding one row — no `switch` to rewrite.


```

dir = opendir(path);                                /* NULL on failure, errno set */
if (dir == NULL) { print_errno("ERROR: Unable to open directory."); return; }

while ((entry = readdir(dir)) != NULL) {
    if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name, "..") == 0)
        continue;                                /* skip . and .. */
    /* build "dir/name", call lstat_and_type() → FILE / DIRECTORY / ... */
    printf("%-*s %s\n", NAME_COL_WIDTH, entry->d_name, type_buf);
}
closedir(dir);

```

Key points:

- `opendir()` returns a `DIR*` handle (like `fd`, but for directories).
- `readdir()` returns **one entry per call** as a `struct dirent` — the loop

keeps calling until it returns `NULL`. `d_name` = filename, `d_ino` = inode.

- `.` (this directory) and `..` (parent) are **real entries** created by the

kernel — we skip them for a clean display.

- Before printing each entry's type we call our helper `lstat_and_type()`

(from `metadata.c`) which uses `lstat()` — because `d_type` in `dirent` is not reliable on all filesystems (`DT_UNKNOWN`).

- **Every path is closed:** `closedir(dir)` in ALL cases — success and error.

Output looks like:

NAME	TYPE
notes.txt	FILE
ProjectFiles	DIRECTORY

Option 3 — Change Directory (`navigation.c`)

Syscall: `chdir()`

```

if (chdir(path) != 0) {
    print_errno("ERROR: Unable to access directory.");
    return;
}

getcwd(cwd, sizeof(cwd));    /* verify + display new location */

```

How it works: we ask the kernel to make `path` the process's working directory. The kernel validates the path (`ENOENT` if missing, `ENOTDIR` if it's a file, `EACCES` if no permission). After

success we call `getcwd()` to *prove* the change took effect.

Viva trap: `chdir()` changes the directory of *OUR PROCESS* only — the shell that launched us is unaffected. Each process has its own working directory.

Option 4 — Create File (`file_operations.c`)

Syscalls: `open()`, `close()`

```
fd = open(path, O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (fd < 0) { print_errno("Unable to create file."); return; }

printf("File opened successfully.\n");
printf("File Descriptor: %d\n", fd);
close(fd);
```

The flags (memorize all three):

- `O_WRONLY` — open for **write** only
- `O_CREAT` — **create** the file if it doesn't exist
- `O_TRUNC` — if it **does** exist, cut it to zero length

The `0644`: the permission argument applied at creation — `6 = rw-` (owner), `4 = r--` (group), `4 = r--` (others). The kernel filters it through the process **umask**, so the final result is typically `rw-r--r--`.

Why close immediately? `open()` already created the file on disk — the fd was only needed for the creation. Not closing leaks descriptors: a process can only have a limited number open (typically 1024).

Option 5 — Read File (`file_operations.c`)

Syscalls: `open()`, `read()`, `close()`

The **canonical read loop** — this exact pattern is what examiners love:

```

fd = open(path, O_RDONLY);

for (;;) {
    bytes_read = read(fd, buffer, sizeof(buffer));
    if (bytes_read < 0) { print_errno("Unable to read file."); close(fd); return; }
    if (bytes_read == 0) break;          /* 0 = END OF FILE */
    fwrite(buffer, 1, bytes_read, stdout);
    total += bytes_read;
}
close(fd);

```

The three possible outcomes of read():

1. `> 0` — number of bytes actually read (may be FEWER than requested — a

partial read is normal, not an error, so we just loop again)

1. `0` — **end of file** (that's how we know to stop!)
2. `-1` — error, check `errno` (`EISDIR` if it's a directory, `EACCES...`)

Empty file: the loop's first `read()` returns 0 immediately → we print `[NOTICE] File is empty (0 bytes)`.

Option 6 — Write to File (`file_operations.c`)

Syscalls: `open()`, `write()`, `close()`

```

fd = open(path, O_WRONLY | O_CREAT | O_APPEND, 0644);

p = text; remaining = strlen(text);
while (remaining > 0) {
    written = write(fd, p, remaining);
    if (written < 0) {
        if (errno == EINTR) continue; /* interrupted by signal: retry */
        print_errno("Unable to write to file."); close(fd); return;
    }
    p += written;          /* advance past what was written */
    remaining -= written;  /* keep going until nothing left */
}
write(fd, "\n", 1);      /* newline so appends stay on separate lines */
close(fd);

```

Key points:

- `O_APPEND` — the kernel **automatically** moves the offset to the end of the

file before every `write()*`, atomically. Existing content is never overwritten.

- **Partial write handling:** `write()` may transfer fewer bytes than asked.

A correct program loops until everything is written. (This is a favorite viva question: "what if `write()` writes only half?")

Option 7 — Create Directory (`directory_operations.c`)

Syscall: `mkdir()`

```
if (mkdir(path, 0755) != 0) { print_errno("Unable to create directory."); return; }
```

- `0755` = `rwxr-xr-x` — owner: full access; group/others: read + traverse.
 - Existing directory → `errno` `EEXIST` → "Reason: File exists."
 - The kernel automatically creates `.` and `..` inside.
-

Option 8 — Remove Directory (`directory_operations.c`)

Syscall: `rmdir()`

```
if (strcmp(path, "/") == 0 || strcmp(path, ".") == 0 || strcmp(path, "..") == 0) {  
    msg_error("Refusing to remove \"/\", \".\" or \"..\"."); /* safety filter */  
    return;  
}  
  
if (rmdir(path) != 0) { print_errno("Unable to remove directory."); return; }
```

- `rmdir()` only works on an **EMPTY** directory (only `.` and `..` inside);

otherwise `errno` = `ENOTEMPTY` → "Reason: Directory not empty."

- The safety filter is **defence in depth**: `rmdir` itself would refuse, but

we politely reject obviously dangerous names before even trying.

Option 9 — File Information / Metadata (`metadata.c`) ★ most important

Syscalls: `lstat()`, `readlink()`

```

if (lstat(path, &file_stat) != 0) { print_errno("Unable to get file information."); return; }

if (S_ISREG(file_stat.st_mode))      file_type = "Regular File";
else if (S_ISDIR(file_stat.st_mode)) file_type = "Directory";
else if (S_ISLNK(file_stat.st_mode)) file_type = "Symbolic Link";
/* ... also CHR, BLK, FIFO, SOCKET macros ... */

```

struct stat — the fields we display:

Field	Meaning
<code>st_size</code>	size in bytes
<code>st_ino</code>	inode number
<code>st_mode</code>	packed bits: file type + 9 permission bits
<code>st_nlink</code>	number of hard links
<code>st_uid</code> / <code>st_gid</code>	numeric owner/group (converted to names via <code>getpwuid</code> / <code>getgrgid</code>)
<code>st_atime</code>	last a ccess time
<code>st_mtime</code>	last m odification time (content changed)
<code>st_ctime</code>	last c hange of the inode itself (permissions, rename...) — NOT creation time!

Permission string — built bit by bit from `st_mode`:

```

(mode & S_IRUSR) ? 'r' : '-' /* user read */
(mode & S_IWUSR) ? 'w' : '-' /* user write */
(mode & S_IXUSR) ? 'x' : '-' /* user execute */
/* ...same for S_IRGRP/S_IWGRP/S_IXGRP and S_IROTH/S_IWOTH/S_IXOTH */

```

Leading char = type: `d` directory, `l` symlink, `-` regular file.

stat() vs lstat() (classic viva):

- `stat()` **follows** symbolic links — reports the TARGET.
- `lstat()` reports the **LINK ITSELF** if the path is a symlink.
- For a symlink we also call `readlink()` to show where it points:

Points To : `-> /some/target`.

S_ISREG / **S_ISDIR** / **S_ISLNK** are MACROS (not functions) that test the file type bits inside `st_mode`.

Option 10 — File Offset / Random Access (`file_operations.c`)

Syscalls: `open()`, `lseek()`, `read()`, `close()`

```
fd = open(path, O_RDONLY);
/* ... validate offset with parse_int(), reject negatives ... */

new_offset = lseek(fd, (off_t)offset, SEEK_SET);
if (new_offset == (off_t)-1) { print_errno("lseek() failed."); ... }

printf("File offset moved successfully. New offset: %ld\n", (long)new_offset);
bytes_read = read(fd, buffer, 200); /* read FROM THE NEW POSITION */
```

`lseek()` — the three whence values:

whence	offset is counted from
<code>SEEK_SET</code>	beginning of file
<code>SEEK_CUR</code>	current position
<code>SEEK_END</code>	end of file

Return value: the NEW offset from the beginning (that's how you can also find file size: `lseek(fd, 0, SEEK_END)`), or `(off_t)-1` on error (e.g. negative result → `EINVAL`).

Viva gold: *`lseek()` only changes the bookkeeping — the file offset stored in the kernel's open-file-description. It moves NO data and touches NO disk. The next `read()/write()` simply continues from the new position. That is why it's called "random access".*

Our validation: `parse_int()` rejects "abc", "12abc", overflow; then we reject negatives ourselves; offset beyond EOF is legal — `read()` just returns 0 ("at end of file").

Option 11 — Copy File (added feature)

Syscalls: `open()` ×2, `fstat()`, `read()`, `write()`, `close()` ×2

The 6-step recipe in `copy_file()`:

```

1. open() SOURCE O_RDONLY          → src_fd
2. fstat(src_fd)                  → size + permission bits
   (fstat = stat by DESCRIPTOR, not path – no race window)
3. if S_ISDIR → refuse ("copy works on files only")
4. open() DESTINATION O_WRONLY|O_CREAT|O_TRUNC with source's mode bits
5. read()/write() loop with partial-read AND partial-write handling,
   retrying on EINTR
6. close() BOTH – the destination's close() can report deferred
   write errors that write() already "succeeded" on

```

Also guards: empty names, source == destination.

Viva: *we built the copy ourselves from primitives — it is a genuine kernel-level copy, not a `system("cp ...")` shell-out.*

Option 12 — Rename / Move (added feature)

Syscall: `rename()`

```

if (rename(old_path, new_path) != 0) { print_errno("Unable to rename."); return; }
printf("Renamed: %s -> %s\n", old_path, new_path);

```

One atomic kernel operation:

- No data is copied — only the directory entry (name → inode mapping) changes.
- If the new name exists, it is **silently replaced** (POSIX semantics).
- It's **atomic**: observers never see a half-renamed file.
- Works on **directories** too (we tested OldDir → NewDir).
- Errors: ENOENT (missing), EACCES, EXDEV (across filesystems — this is why

`mv` falls back to copy+delete between partitions).

PART 7 — THE UTILITIES (utils.c — the shared toolbox)

Function	What it does	Why it matters
<code>read_line(buf, size)</code>	Reads one line with <code>fgets()</code> , strips <code>\n</code> and <code>\r</code> , drains overlong input	Buffer-overflow-proof. <code>gets()</code> is banned — it has NO length limit. Our version can never overflow.
<code>trim(s)</code>	Removes leading/trailing spaces	<code>" 4 "</code> must work as choice 4
<code>parse_int(s, &out)</code>	Strict string→int with <code>strtol()</code>	Rejects <code>"abc"</code> , <code>"12abc"</code> , empty, and overflow (checks <code>errno == ERANGE</code> and <code>INT_MIN/MAX</code>)
<code>prompt_line(prompt, ...)</code>	Print prompt + flush + read one line	<code>fflush(stdout)</code> makes the prompt appear BEFORE blocking on input
<code>print_errno(text)</code>	Prints <code>[ERROR] text + Reason: strerror(errno)</code>	Unified error reporting everywhere
<code>print_perror(text)</code>	<code>perror()</code> — same job, writes to stderr	Both used, as the spec requires
<code>ui_init()</code>	Enables ANSI colors only if <code>isatty(STDOUT_FILENO)</code> and <code>NO_COLOR</code> unset	If output is piped to a file, color codes would look like garbage <code>^[[31m</code> — so they're auto-disabled
<code>msg_success / msg_error / msg_notice</code>	<code>[SUCCESS]</code> green / <code>[ERROR]</code> red / <code>[NOTICE]</code> yellow	The consistent UI language of the whole program

Input validation summary (viva: "how do you prevent crashes?"):

- every menu choice: parsed strictly, range-checked 1–13
- every filename: empty-string check before any syscall
- every offset: strict integer parse + negative check
- every buffer: fixed size (PATH_BUF 4096 / LINE_BUF 1024 / NAME_BUF 256),

always used with `sizeof()` — never a magic number

- EOF (Ctrl-D): treated as clean exit, not a crash

PART 8 — ERROR HANDLING PHILOSOPHY (the grading rubric loves this)

Rule: EVERY system call's return value is checked. No exceptions.

```

if (fd < 0)           /* open() */
if (bytes_read < 0)   /* read() */
if (written < 0)       /* write() */
if (close(fd) != 0)    /* close() */
if (dir == NULL)       /* opendir() */
if (mkdir(...) != 0)   /* mkdir(), rmdir(), chdir(), lseek() == (off_t)-1 */

```

And on every failure path:

1. A human-readable `[ERROR]` message is printed,
2. `errno` is translated via `strerror()` ("Reason: ..."),
3. **already-opened resources are closed** (see: every error branch calls

`close(fd)` / `closedir(dir)` before returning),

1. control returns safely to the menu — the program **never crashes**.

Resource discipline: every `open()` has exactly one `close()` on every path (success AND failure); every `opendir()` has a `closedir()`. That's what the closing banner means by "no leaked kernel resources".

PART 9 — THE MAKEFILE (what happens when you type `make`)

```

CC      = gcc
CFLAGS  = -std=c11 -Wall -Wextra -Wpedantic -O2 -Iinclude -Isrc
TARGET  = build/explorer

$(TARGET): $(OBJECTS)
    $(CC) $(OBJECTS) -o $@          # link
$(BUILDDIR)/%.o: $(SRCDIR)/%.c ...
    $(CC) $(CFLAGS) -c $< -o $@    # compile each .c → .o

```

Step by step:

1. Each `.c` in `src/` is compiled separately into an object file `build/*.o`
- (`-c` = compile only, don't link).

1. All object files are **linked** together into the single executable

`build/explorer`.

1. `-Iinclude` tells the compiler where to find our headers.
2. **Warnings:** `-Wall -Wextra -Wpedantic` turn on everything — our project

compiles with **ZERO warnings**.

1. `make` is smart: it rebuilds only files whose sources changed.

2. `make clean` deletes the whole `build/` folder.

Direct compile without make (one line):

```
gcc -std=c11 -Wall -Wextra -Wpedantic -O2 -Iinclude -Isrc src/*.c -o build/explorer
```

PART 10 — HOW TO RUN (the exact steps)

```
# 1. open WSL / Ubuntu terminal
# 2. go to the project folder (adjust path to yours):
cd "/mnt/d/2nd yr Odd_sem/(OSSP)OPERATING SYSTEMS AND SYSTEMS PROGRAMMING/Project_OSSP/LinuxFi

# 3. build + run:
make run

# 4. in the menu: type a number 1-13, press Enter. 13 = exit.
```

Extras:

```
make clean          # remove build folder
bash tests/run_demo.sh  # full automated demo in /tmp (nothing touched in project)
```

Tools needed: `gcc`, `make` (Ubuntu: `sudo apt install build-essential`).

PART 11 — FULL DEMO SCRIPT (practice this for the review!)

Do this in a scratch folder so you create files safely:

```
cd /tmp && mkdir -p demo && cd demo
"/mnt/d/2nd yr Odd_sem/(OSSP)OPERATING SYSTEMS AND SYSTEMS PROGRAMMING/Project_OSSP/LinuxFiles
```

Then, in order (say out loud which syscall each step demonstrates):

#	Press	Enter	Shows	Syscall
1	1	—	Current Directory	getcwd()
2	2	<i>(Enter for current)</i>	Listing with types	opendir/readdir/closedir
3	7	ProjectFiles	Directory created	mkdir()
4	2	—	ProjectFiles now listed (DIRECTORY)	readdir again
5	4	notes.txt	File created, fd 3 printed	open() O_CREAT
6	6	notes.txt then Hello from Linux system calls	Content written	open+write
7	6	notes.txt, Second line via O_APPEND	Append works	O_APPEND
8	5	notes.txt	Both lines shown, byte count	read()
9	9	notes.txt	Size, permissions, inode, 3 timestamps	stat()
10	9	ProjectFiles	File Type: Directory	stat on dir
11	10	notes.txt, 6	Offset moved to 6, shows content from position 6	lseek()
12	10	notes.txt, 9999	"at (or beyond) end of file"	lseek edge case
13	11	notes.txt then backup.txt	Copied, two fds shown	open/fstat/read/write/close
14	12	backup.txt then renamed.txt	Renamed instantly	rename()
15	5	nosuchfile.txt	ERROR + Reason: No such file or directory	errno = ENOENT
16	4	ProjectFiles/inner.txt	File created INSIDE ProjectFiles (open O_CREAT works with a path)	open()
17	8	ProjectFiles	ERROR: Directory not empty (ENOTEMPTY!)	errno = ENOTEMPTY
18	5	ProjectFiles/inner.txt	(optional) proves the file exists	read()
19	99	—	Invalid menu choice — no crash	input validation
20	13	—	Clean exit message	—

To show rmdir SUCCESS too: press `7`, enter `EmptyDir` (creates it), then press `8`, enter `EmptyDir` → "Directory removed successfully." (Our project has no unlink/delete-file option by design — the core spec doesn't ask for one — so demo ENOTEMPTY with `ProjectFiles` and rmdir-success with `EmptyDir`.)

The killer demo for lseek (step 11): write `ABCDEFGHIJ` in a file, then lseek to offset 4 → it prints `EFGHIJ`. Visual proof the offset moved.

PART 12 — HOW WE TESTED IT

18 scripted scenarios (full table with real outputs: `tests/test_cases.md`). Method: feed a scripted list of inputs into the program (`./build/explorer < demo_input.txt`) inside a throwaway `/tmp` directory, then verify every message. Highlights:

- **TEST 7** read missing file → `[ERROR] ... Reason: No such file or directory.`
- **TEST 5** create existing file → succeeds with `O_TRUNC` (documented)
- **TEST 10** mkdir existing → `Reason: File exists.`
- **TEST 12** rmdir non-empty → `Reason: Directory not empty.`
- **TEST 14** lseek valid offset → content printed from that byte
- **TEST 15** invalid input (`abc` , `99` , `-5`) → clean error, no crash
- Copy verified **byte-exact** by reading the copy back (63 bytes identical)
- Copy preserves permissions (`-rw-r--r--`), rejects directories and self-copy
- Rename tested on files AND directories, and over an existing name

Build quality: `gcc -std=c11 -Wall -Wextra -Wpedantic` → **0 warnings**.

Honestly not verified (say this if asked): `valgrind` memory checking and `strace` syscall tracing were not available in our environment — rename atomicity was verified behaviorally, not by tracing.

PART 13 — VIVA QUESTIONS & CRISP ANSWERS

1. What is a system call? A function that switches the CPU from user mode to kernel mode so the kernel performs a privileged operation on behalf of the program, then returns a result. Our project's every operation is one.

2. What is a file descriptor? A small integer index into the kernel's per-process open-file table, identifying an open file. `0/1/2` = `stdin/stdout/stderr`; our first open usually gets `3`.

3. What does open() return? The lowest free descriptor number on success; `-1` on failure with `errno` set.

4. Difference between open() and fopen()? `open()` is the low-level POSIX system-call layer returning an `int fd`, unbuffered. `fopen()` is a C library function returning `FILE*`, with internal user-

space buffering. We use `open()` to demonstrate kernel interaction.

5. What does `read()` do? Copies up to `N` bytes from the file (at its current kernel-side offset) into our buffer. Returns bytes read, 0 at EOF, -1 on error. Partial reads are normal.

6. What does `write()` do? Copies bytes from our buffer into the file at the current offset. May write fewer than requested (partial write) — we loop until all bytes are written.

7. Why is `close()` required? It releases the descriptor back to the kernel and flushes kernel buffers. Processes have a finite descriptor limit; every open must be matched by a close on all code paths. `close()` can even report deferred write errors.

8. What is `lseek()`? Repositions the file offset associated with a descriptor (`SEEK_SET/CUR/END`). It changes no data — the next read/write simply continues from the new position. Returns the new offset or `(off_t)-1`.

9. What is `stat()`? A system call that fills a `struct stat` with the file's inode metadata: type, size, permissions, inode, owner, and the `atime/mtime/ctime` timestamps.

10. What is an inode? The filesystem data structure holding a file's metadata and data-block pointers. Names live in directories (name→inode mapping), not in the inode.

11. Difference between file and directory? A directory is itself a special file whose content is a table of name→inode entries. Files hold data; directories organize names.

12. What are file permissions? 9 bits in `st_mode`: `rw` for user, group, others. We build the string `rw-r--r--` by testing `S_IRUSR`, `S_IWUSR`, ... macros. Creation modes are filtered by the `umask`.

13. What are `opendir()` and `readdir()`? `opendir()` opens a directory stream (`DIR*`); each `readdir()` call returns the next entry as `struct dirent` (`d_name`, `d_ino`), NULL at the end. `closedir()` releases it.

14. What does `mkdir()` do? Creates a directory with given permissions (filtered by `umask`); the kernel creates the `.` and `..` entries automatically. Fails with `EEXIST` if present.

15. What does `rmdir()` do? Removes an EMPTY directory only — `ENOTEMPTY` otherwise. The kernel guarantees the directory contains nothing but `.` and `..`.

16. What is `errno`? A global integer set by system calls on failure, giving the REASON for the failure (`ENOENT`, `EACCES`, `EISDIR`, `ENOTEMPTY`...). Checked after the call indicates failure by its return value.

17. Why is `perror()` useful? It prints your text + ": " + the `errno` message in one call (to `stderr`). We also use `strerror(errno)` for formatted output on `stdout`.

18. What happens when a system call fails? It returns -1 (or NULL), sets `errno`, and performs no action. Our code checks the return value, prints the message + reason, closes anything already open, and returns to the menu.

19. What is a kernel? The privileged core of the OS that manages CPU, memory, devices, and the filesystem. User programs may only reach it through system calls.

20. How does the user program interact with the Linux kernel? Via system calls: arguments are placed in registers, a trap instruction switches to kernel mode, the kernel validates and performs the operation, and returns the result to user mode. (Explain with the Part 3 diagram.)

Bonus — added features: 21. Why is `rename()` atomic? It's a single syscall that updates the directory entry mapping; there is no intermediate state where the file is "half-renamed", unlike a copy+delete approach.

22. Why `fstat()` instead of `stat()` in `copy`? `fstat()` queries by descriptor, so the file we inspected is guaranteed to be the one we have open — closes the race window between opening and stat-ing by path.

PART 14 — GLOSSARY (30-second definitions)

- **User space / kernel space** — where normal programs run vs. where the kernel runs; the boundary is crossed only by system calls.

- **POSIX** — the portable UNIX API standard; Linux implements it. Our

`_POSIX_C_SOURCE 200809L` selects the 2008 edition.

- **Mode switch / trap** — the CPU transition into kernel mode that a syscall triggers.

- **umask** — per-process bits that are REMOVED from requested creation permissions (why `0644` request → `rw-r--r--` result).

- **O_APPEND** — flag making every `write()` atomically start at end-of-file.

- **Partial read/write** — a syscall transferring fewer bytes than requested; normal behavior; handled by looping.

- **EINTR** — `errno` meaning "interrupted by a signal"; the correct response is to retry the call (we do).

- **Hard link** — another directory entry pointing to the SAME inode.

- **Symlink** — a separate small file containing a PATH; `stat()` follows it, `lstat()` doesn't.

- **Current working directory** — the per-process directory that relative paths resolve from; `getcwd()` reads it, `chdir()` changes it.

- **Directory stream** — the `DIR*` handle from `opendir()` that `readdir()` advances through.

- **Makefile target** — a named goal (`all`, `run`, `clean`) with a recipe.

PART 15 — ONE-PAGE CHEAT SHEET (if you remember nothing else)

KERNEL = boss of the hardware. You talk to it ONLY via SYSTEM CALLS.

FD = ticket number the kernel gives you for an open file (0,1,2 reserved;
first open = usually 3 – print it, never assume it).

errno = WHY the last syscall failed. strerror(errno) = human text.

open() → fd O_CREAT create, O_TRUNC empty it, O_APPEND end-of-file writes,
 O_RDONLY read, O_WRONLY write, mode 0644 → rw-r--r--

read() → >0 bytes, 0 = EOF (stop!), -1 = error. Loop until 0.

write() → may write LESS than asked → loop until done.

close() → EVERY open() must be closed on every path.

lseek() → moves the offset bookkeeping, moves NO data.
 SEEK_SET from start / SEEK_CUR here / SEEK_END from end.

stat() → struct stat: size, inode, mode, 3 timestamps. Follows symlinks.

lstat() → same but shows the LINK itself. S_ISREG/S_ISDIR/S_ISLNK macros.

opendir/readdir/closedir → one entry per call, NULL at end, skip "." and ".."

mkdir() → 0755. rmdir() → EMPTY dirs only (ENOTEMPTY).

rename() → ONE atomic syscall, just re-points the name, no data copied.

ALWAYS → check return value; print [ERROR] + Reason: strerror(errno);
 close what you opened; never crash on bad input.

fgets() not gets(); strtol() not atoi(); snprintf() not sprintf().

End of guide. Every claim here maps to real code in `src/` and real verified runs recorded in `tests/test_cases.md` — you can demo and defend all of it.